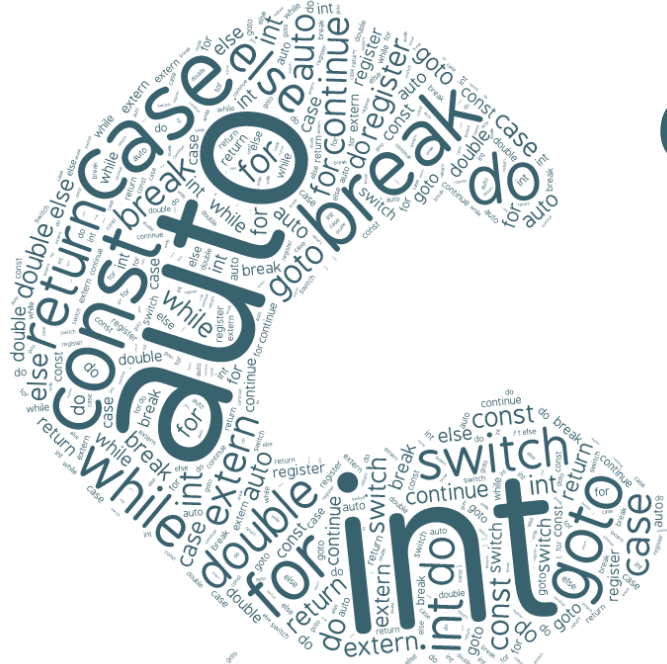




chapter 9

查找

主讲人：周瑜

9.1 基本概念

9.2 静态查找表

9.3 动态查找表

9.4 哈希表

教材第8、11和12章省略，因《操作系统》课程会涉及。

查找表

由**同一类型**的数据元素(或记录)构成的**集合**。

由于“集合”中的数据元素之间存在着**松散的关系**，因此查找表是一种应用灵便的结构。

对查找表经常进行的操作：

- 1) **查询**某个“**特定的**”数据元素是否在查找表中；
- 2) **检索**某个“**特定的**”数据元素的各种属性；
- 3) 在查找表中**插入**一个数据元素；
- 4) 从查找表中**删去**某个数据元素。

查找表可分为两类：

静态查找表 仅作查询和检索操作的查找表。

动态查找表 有时在查询之后，还需要将“查询”结果为“不在查找表中”的数据元素插入到查找表中；或者，从查找表中删除其“查询”结果为“在查找表中”的数据元素。

关键字

是数据元素（或记录）中某个**数据项**的值，用以**标识**（识别）一个数据元素（或记录）。

若此关键字可以识别**唯一的**一个记录，则称之为“主关键字”。

若此关键字能识别**若干**记录，则称之为“次关键字”。

查找（搜索、Search）

- 所谓查找，就是在数据集合中寻找满足某种条件的数据对象。
- 查找的结果通常有两种可能：

查找成功，即找到满足条件的数据对象。报告该对象在结构中的位置，还可进一步给出该对象中的具体信息。

查找不成功，或搜索失败。报告失败标志、失败位置等。

通常称用于搜索的数据集合为搜索结构，它是由同一数据类型的对象(或记录)组成。

如何进行查找？

查找的方法取决于**查找表的结构**。

由于查找表中的数据元素之间不存在明显的组织规律，因此不便于查找。

为了提高查找的效率，需要在查找表中的元素之间人为地附加某种确定的关系，换句话说，用另外一种结构来表示查找表。

衡量一个搜索算法的时间效率的标准

在搜索过程中关键码的平均比较次数，这个标准也称为平均搜索长度 $ASL(Average\ Search\ Length)$ ，通常它是搜索结构中对象总数 n 或文件结构中物理块总数 n 的函数。

另外衡量一个搜索算法还要考虑算法所需要的存储量和算法的复杂性等问题。

假设静态查找表的顺序存储结构为

```
typedef struct {  
    ElemType *elem; // 数据元素存储空间基址，建表时按实际长度分配，0号单元留空  
    int length; // 表的长度  
} SSTable;
```

数据元素类型的定义为：

```
typedef struct {  
    keyType key; // 关键字域  
    ... .. // 其它属性域  
} ElemType ; TElemType ;
```

9.1 基本概念

9.2 静态查找表

9.3 动态查找表

9.4 哈希表

一、顺序查找表

- 设若表中有 n 个对象，则顺序查找从表的先端开始，**顺序用各对象的关键码与给定值 x 进行比较**，直到找到与其值相等的对象，则查找成功，给出该对象在表中的位置。
- 若整个表都已检测完仍未找到关键码与 x 相等的对象，则查找失败。给出失败信息。

ST.elem

64	21	37	88	19	92	05	64	56	80	75	13	
0	1	2	3	4	5	6	7	8	9	10	11	

key=64

ST.Length

k

i

假设给定值 $e=64$,
要求 $ST.elem[k] = e$, 问: $k = ?$

```
int Search_Seq(SSTable ST, KeyType key)
{ // 在顺序表ST中顺序查找其关键字等于key的数据元素。
  // 若找到，则函数值为该元素在表中的位置，否则为0。
  ST.elem[0].key = key;    // “哨兵”
  for (i=ST.length; ST.elem[i].key!=key; --i); // 从后往前找
  return i;                // 找不到时，i为0
} // Search_Seq
```

所查找的记录序号

关键字需比较次数

n

1

n-1

2

n-2

3

...

...

2

n-1

1

n

平均比较次数 $(1+2+3+\dots+n-1+n)/n = (n+1)/2$

查找算法的**平均查找长度** (Average Search Length)约为表长的一半。

二、有序查找表

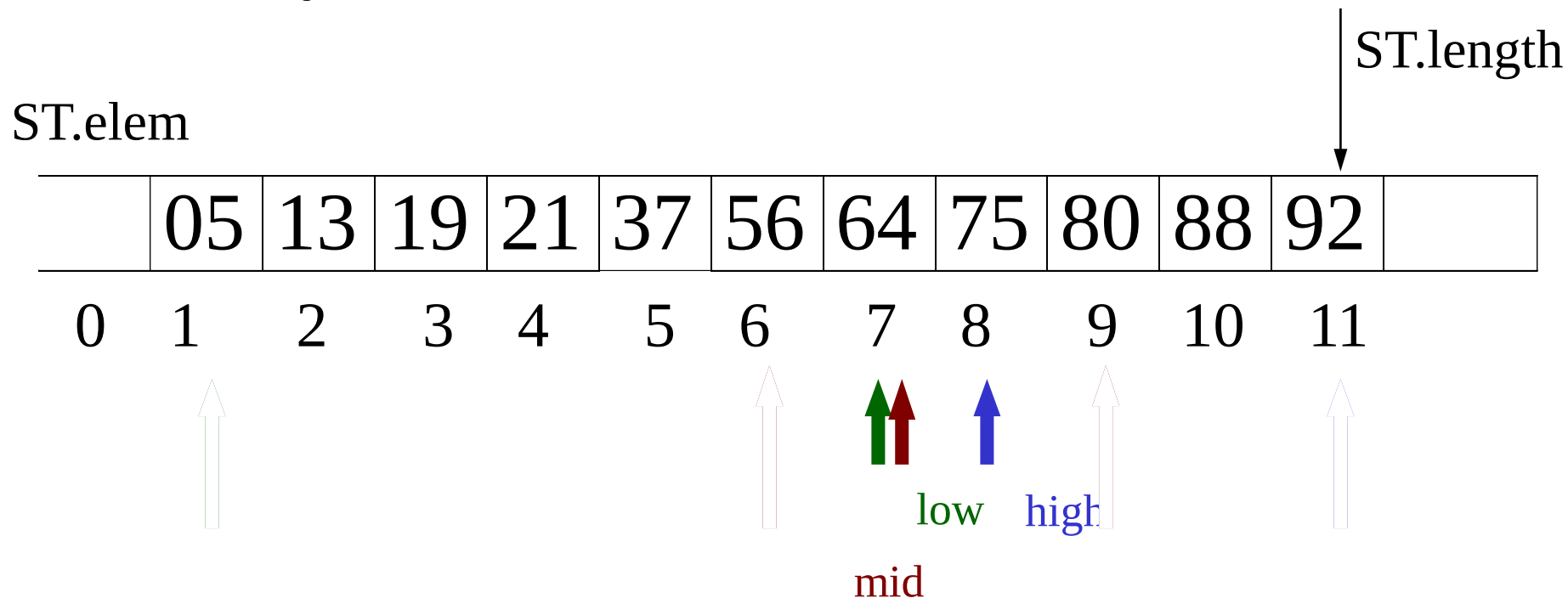
上述顺序查找表的查找算法简单，但**平均查找长度较大**，即查找效率太低，特别不适用于表长较大的查找表。

若以**有序表**表示静态查找表，则查找过程可以基于“**折半**”进行。

基于有序顺序表的折半搜索

- 设 n 个对象存放在一个有序顺序表中，并按其关键字从小到大排好了序。
- 采用折半查找时，先求位于查找区间正中的对象的下标 mid ，用其关键码与给定值 x 比较：
 - $Elem[mid].Key = x$ ，查找成功；
 - $Elem[mid].Key > x$ ，把查找区间缩小到表的前半部分，再继续进行折半查找
 - $Elem[mid].Key < x$ ，把查找区间缩小到表的后半部分，再继续进行折半查找
- 每比较一次，查找区间缩小一半。如果查找区间已缩小到一个对象，仍未找到想要查找的对象，则查找失败。

例如：**key=64** 的查找过程如下：

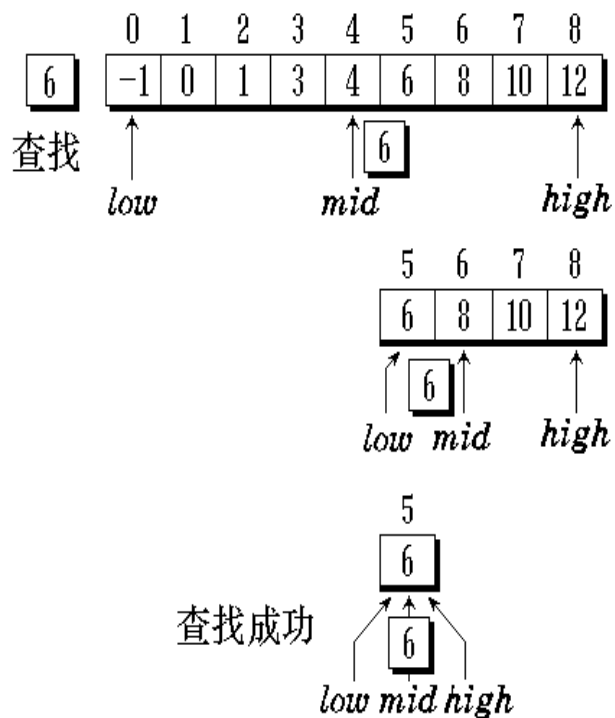


low 指示查找区间的下界

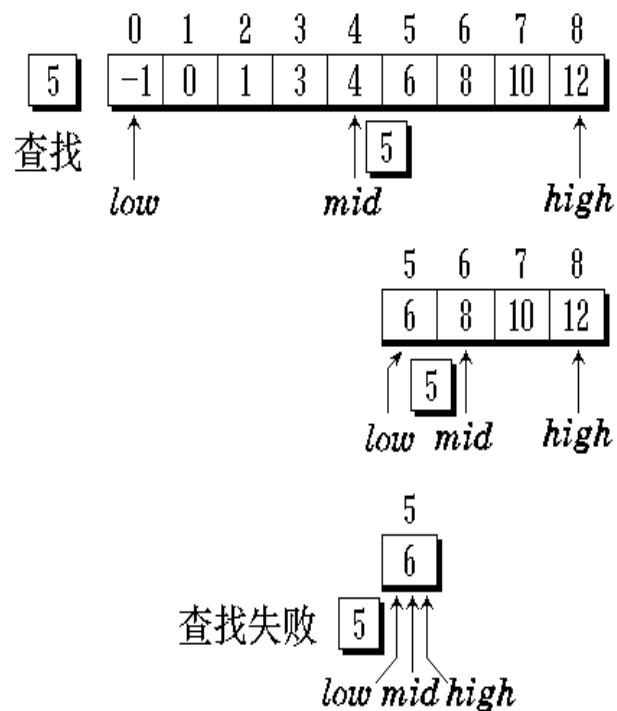
high 指示查找区间的上界

mid = $(low+high)/2$

通过比ST.elem[mid].key与key的大小来逐步缩小查找范围



搜索成功的例子



搜索失败的例子

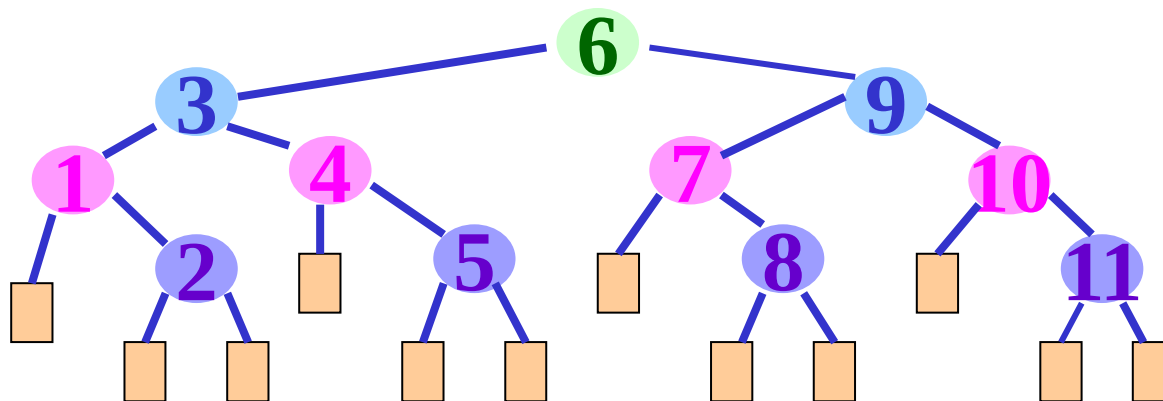
```
int Search_Bin ( SSTable ST , KeyType key ) {  
    low = 1; high = ST.length;    // 置区间初值  
    while (low <= high) {        // 折半查找  
        mid = (low + high) / 2;  
        if (EQ (key , ST.elem[mid].key) )  
            return mid;          // 找到待查元素  
        else if ( LT (key , ST.elem[mid].key) )  
            high = mid - 1;      // 继续在前半区间进行查找  
        else low = mid + 1;      // 继续在后半区间进行查找  
    }  
    return 0;                    // 顺序表中不存在待查元素  
} // Search_Bin
```

分析折半查找的平均查找长度

先看一个具体的情况，假设： $n=11$

i	1	2	3	4	5	6	7	8	9	10	11
Ci	3	4	2	3	4	1	3	4	2	3	4

判定树



折半查找过程可用二叉判定树来描述

- 一般情况下，表长为 n 的折半查找的判定树的深度和含有 n 个结点的完全二叉树的深度相同。
- 在有序表中查找记录的过程就是走了一条从根结点到与该记录相应的结点的路径，和给定值进行的比较次数就是该结点在判定树上的层次数。
- 折半查找在查找成功时关键字比较次数不超过树的深度，即： $\lfloor \log_2 n \rfloor + 1$

假设 $n=2^h-1$ 并且查找概率相等

则

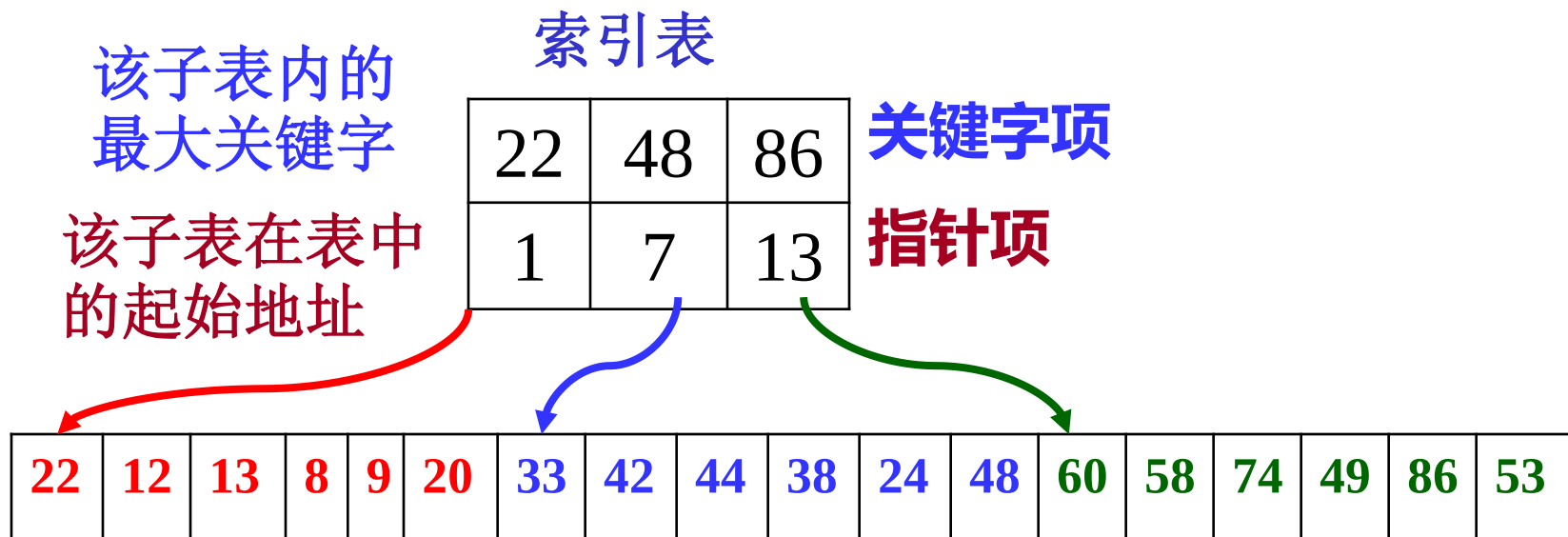
$$ASL_{bs} = \frac{1}{n} \sum_{i=1}^n C_i = \frac{1}{n} \left[\sum_{j=1}^h j \times 2^{j-1} \right] = \frac{n+1}{n} \log_2(n+1) - 1$$

在 $n>50$ 时, 可得近似结果 $ASL_{bs} \approx \log_2(n+1) - 1$

折半查找的效率比顺序查找高, 但折半查找只适用于有序表, 且限于顺序存储结构 (为什么线性链表无法实施折半查找?)

索引顺序表的查找

- 查找方法：分块查找（索引顺序查找）
- 索引表的建立：



- 分块有序原则：第二个子表的所有记录的关键字均大于第一个子表中的最大关键字，……，依此类推。

索引顺序表的查找过程：

- 1) 由索引确定记录所在块（子表）；
- 2) 在顺序表的某个块内进行顺序查找。

可见，索引顺序查找的过程也是一个“缩小区间”的查找过程。

注意：索引可以根据查找表的特点来构造。

因为索引表中的索引项是按关键字有序排列的，因此可用顺序查找，也可用折半查找；而块（子表）中的记录是任意排列的，只能用顺序查找的方法。

索引顺序查找的平均查找长度 = 查找“索引”的平均查找长度
+ 查找“顺序表”的平均查找长度

综合上一节讨论的几种查找表的特性：

	查找	插入	删除
无序顺序表	$O(n)$	$O(n)$	$O(n)$
无序线性链表	$O(n)$	$O(1)$	$O(1)$
有序顺序表	$O(\log n)$	$O(n)$	$O(n)$
有序线性链表	$O(n)$	$O(1)$	$O(1)$

可得如下结论：

- 1) 从查找性能看，最好情况能达 $O(\log n)$ ，此时要求表有序；
- 2) 从插入和删除的性能看，最好情况能达 $O(1)$ ，此时要求存储结构是链表。

9.1 基本概念

9.2 静态查找表

9.3 动态查找表

9.4 哈希表

表的结构是在查找过程中动态生成的（插入操作）

二叉排序树（二叉查找树）

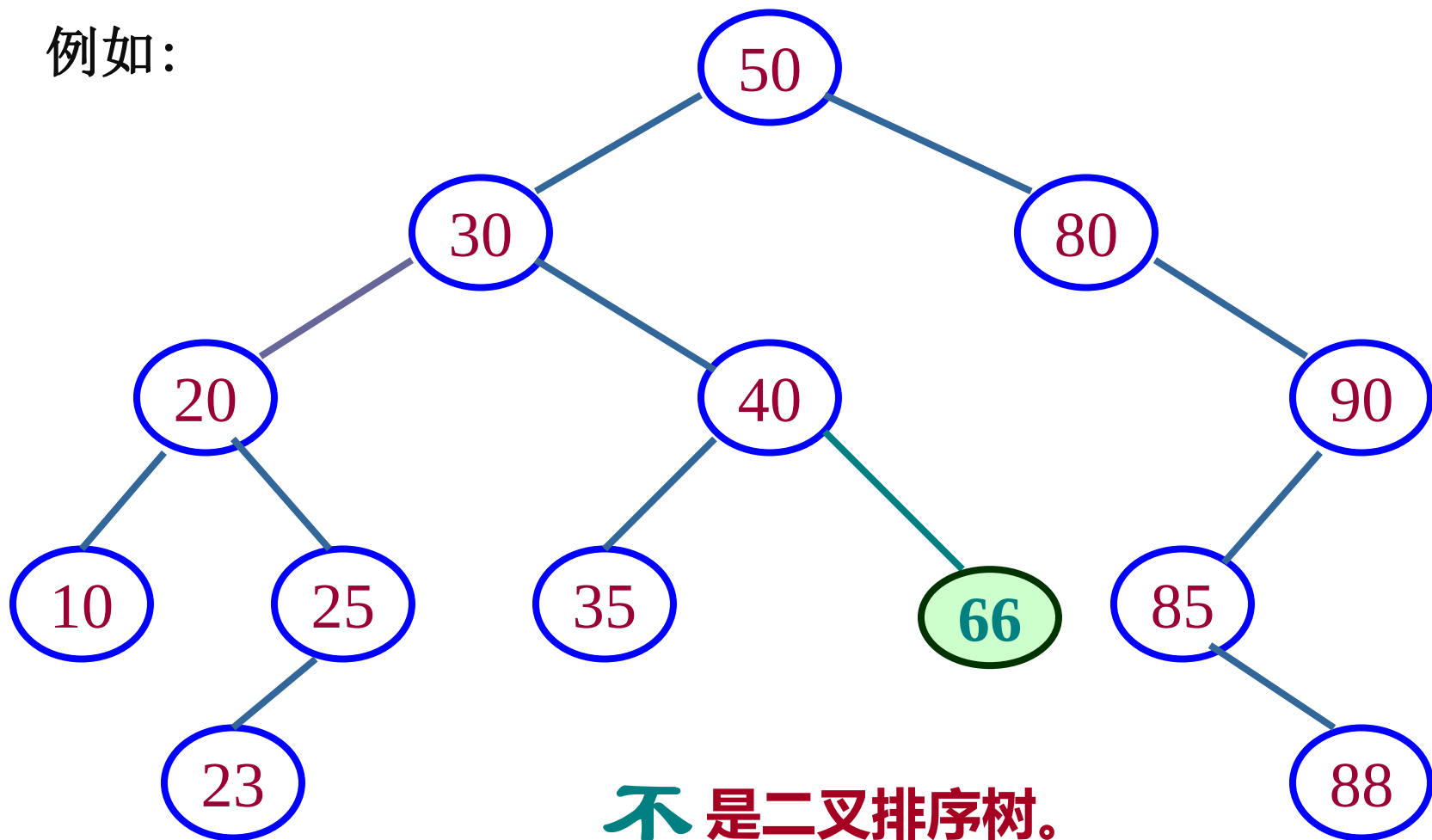
1. 定义
2. 查找算法
3. 插入算法
4. 删除算法
5. 查找性能的分析

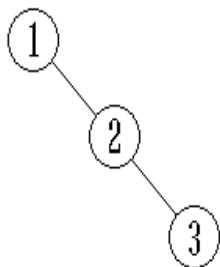
一、二叉排序树（二叉查找树）

二叉排序树或者是一棵空树；或者是具有如下特性的二叉树：

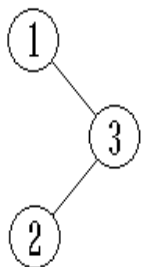
- （1）若它的左子树不空，则左子树上所有结点的值均小于根结点的值；
- （2）若它的右子树不空，则右子树上所有结点的值均大于根结点的值；
- （3）它的左、右子树也都分别是二叉排序树。

例如：

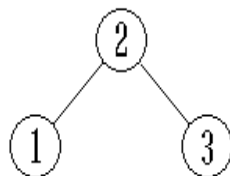




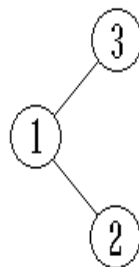
(a)



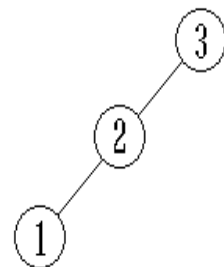
(b)



(c)



(d)



(e)

几个二叉排序树的例子

如果对一棵二叉搜索树进行**中序遍历**，可以按**从小到大**的顺序，将各结点关键字排列起来

通常，取二叉链表作为二叉排序树的存储结构

```
typedef struct BiTNode {  
    // 结点结构  
    TElemType    data;  
    struct BiTNode *lchild, *rchild; // 左右孩子指针  
} BiTNode, *BiTree;
```

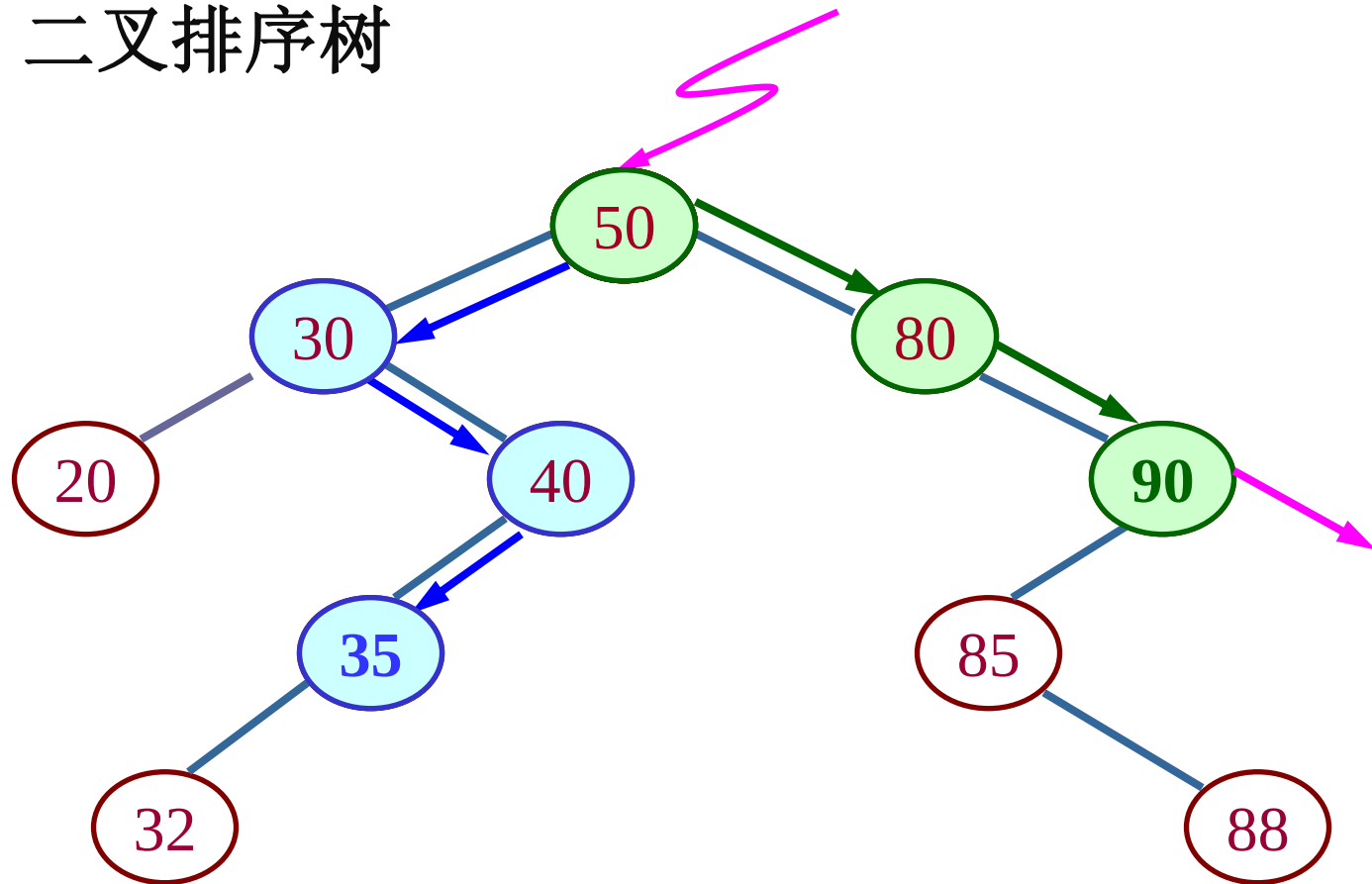
2. 二叉排序树的查找算法

在二叉排序树上进行查找，是一个从根结点开始，沿某一个分支逐层向下进行比较判等的过程。它可以是一个递归的过程。

若二叉排序树为空，则查找不成功；否则，

- 1) 若给定值等于根结点的关键字，则查找成功；
- 2) 若给定值小于根结点的关键字，则继续在左子树上进行查找；
- 3) 若给定值大于根结点的关键字，则继续在右子树上进行查找。

例如：二叉排序树



查找关键字 == 50, 35, 90, 95,

从上述查找过程可见，

在查找过程中，生成了一条查找路径：

从根结点出发，沿着左分支或右分支逐层向下直至关键字等于给定值的结点 一查找成功

从根结点出发，沿着左分支或右分支逐层向下直至指针指向空树为止 一查找不成功

```
BiTree SearchBST (BiTree T, KeyType key, ) {  
    // 在根指针T所指二叉排序树中递归地查找其  
    // 关键字等于key的数据元素，若查找成功，则  
    // 返回指向该数据元素的指针否则表明查找不  
    // 成功，返回空指针NULL  
  
    if( (!T)||EQ(key,T->data.key) )  
        return(T);  
    else if ( LT(key,T->data.key) )  
        return(SearchBST(T->lchild,key));  
    else  
        return (SearchBST(T->rchild,key));  
} // SearchBST
```

```
Status SearchBST ( BiTree T, KeyType key , Bitree f , Bitree &p ) {
```

```
//查找成功p指向该数据元素结点并返回TRUE, 否则p指向查找路
```

```
//径上访问的最后一个结点, 并返回FALSE, f指向T的双亲。
```

```
    if (!T)
```

```
        { p = f; return FALSE; }           // 查找不成功
```

```
    else if ( EQ(key, T->data.key) )
```

```
        { p = T; return TRUE; }           // 查找成功
```

```
    else if ( LT(key, T->data.key) )
```

```
        SearchBST (T->lchild, key, T, p ); // 在左子树中继续查找
```

```
    else
```

```
        SearchBST (T->rchild, key, T, p ); // 在右子树中继续查找
```

```
}
```

3. 二叉排序树的插入算法

- 根据动态查找表的定义，“插入”操作在查找不成功时才进行；
- 若二叉排序树为空树，则新插入的结点为新的根结点；否则，新插入的结点必为一个新的叶子结点，其插入位置由查找过程得到。

- 为了向二叉排序树中插入一个新元素，必须先检查这个元素是否在树中已经存在
- 在插入之前，先使用查找算法在树中检查要插入元素有还是没有。

搜索成功：树中已有这个元素，不再插入。

搜索不成功：树中原来没有关键字等于给定值的结点，把新元素加到查找操作停止的地方。

```
Status Insert BST(BiTree &T, ElemType e )
{ // 当二叉排序树中不存在关键字等于 e.key 的
  // 数据元素时，插入元素值为 e 的结点，并返
  // 回 TRUE; 否则，不进行插入并返回FALSE
  if (!SearchBST ( T, e.key, NULL, p ))
    //查找不成功，插入结点
    {   ...   }
  else
    return FALSE;
} // Insert BST
```

```
s = (BiTree) malloc (sizeof (BiTNode)); // 为新结点分配空间
```

```
s->data = e;
```

```
s->lchild = s->rchild = NULL;
```

```
if ( !p )
```

```
    T = s; // 插入 s 为新的根结点
```

```
else if ( LT(e.key, p->data.key) )
```

```
    p->lchild = s; // 插入 *s 为 *p 的左孩子
```

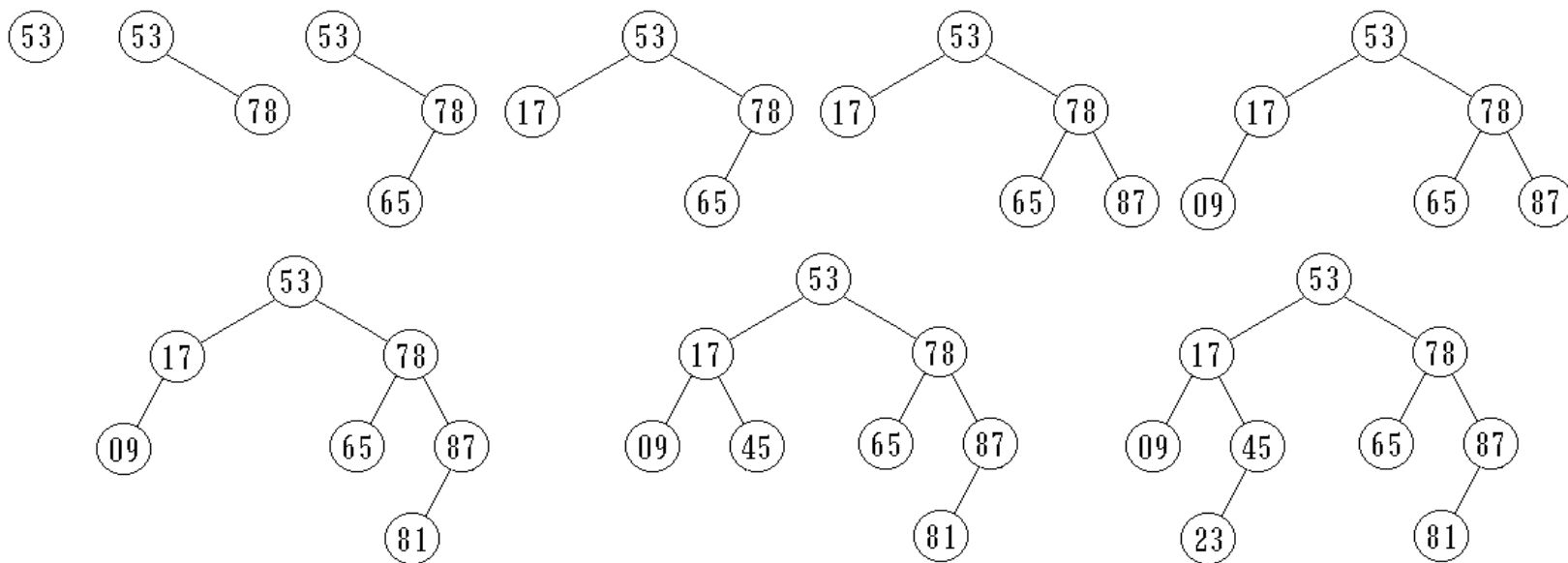
```
else
```

```
    p->rchild = s; // 插入 *s 为 *p 的右孩子
```

```
return TRUE; // 插入成功
```

从空树出发，经过一系列的查找插入操作后，可生成一棵二叉排序树

输入数据序列 { 53, 78, 65, 17, 87, 09, 81, 45, 23 }

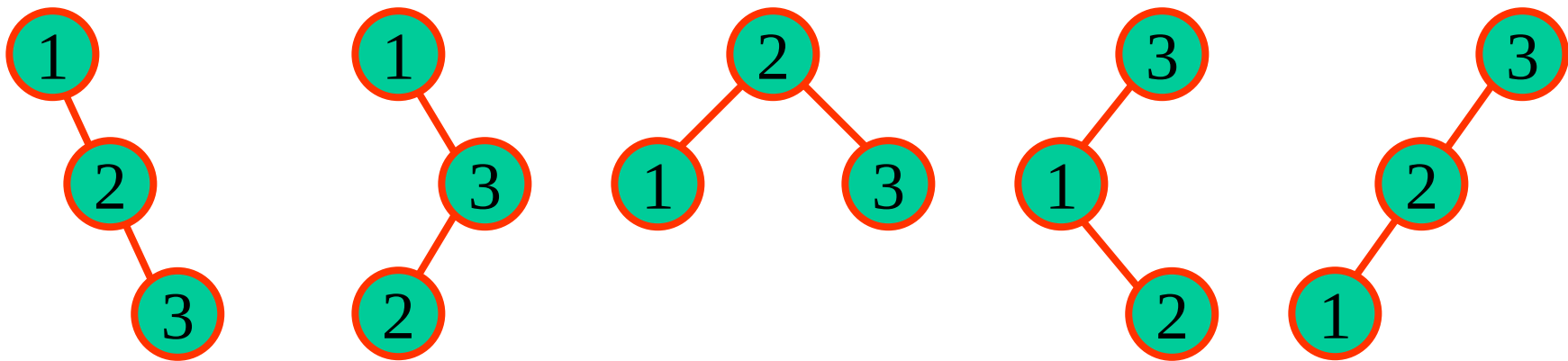


同样3个数据{1,2,3}，输入顺序不同，建立起来的二叉搜索树的形态也不同。这直接影响到二叉搜索树的搜索性能。

如果输入序列选得不好，会建立起一棵单支树，使得二叉搜索树的高度达到最大，这样必然会降低搜索性能

{2, 1, 3}

{1, 2, 3} {1, 3, 2} {2, 3, 1} {3, 1, 2} {3, 2, 1}



4. 二叉排序树的删除算法

和插入相反，删除在查找成功之后进行，并且要求在删除二叉排序树上某个结点之后，**仍然保持二叉排序树的特性**。

在二叉排序树中删除一个结点时，必须将因删除结点而断开的二叉链表重新链接起来，同时确保二叉排序树的性质不会失去。

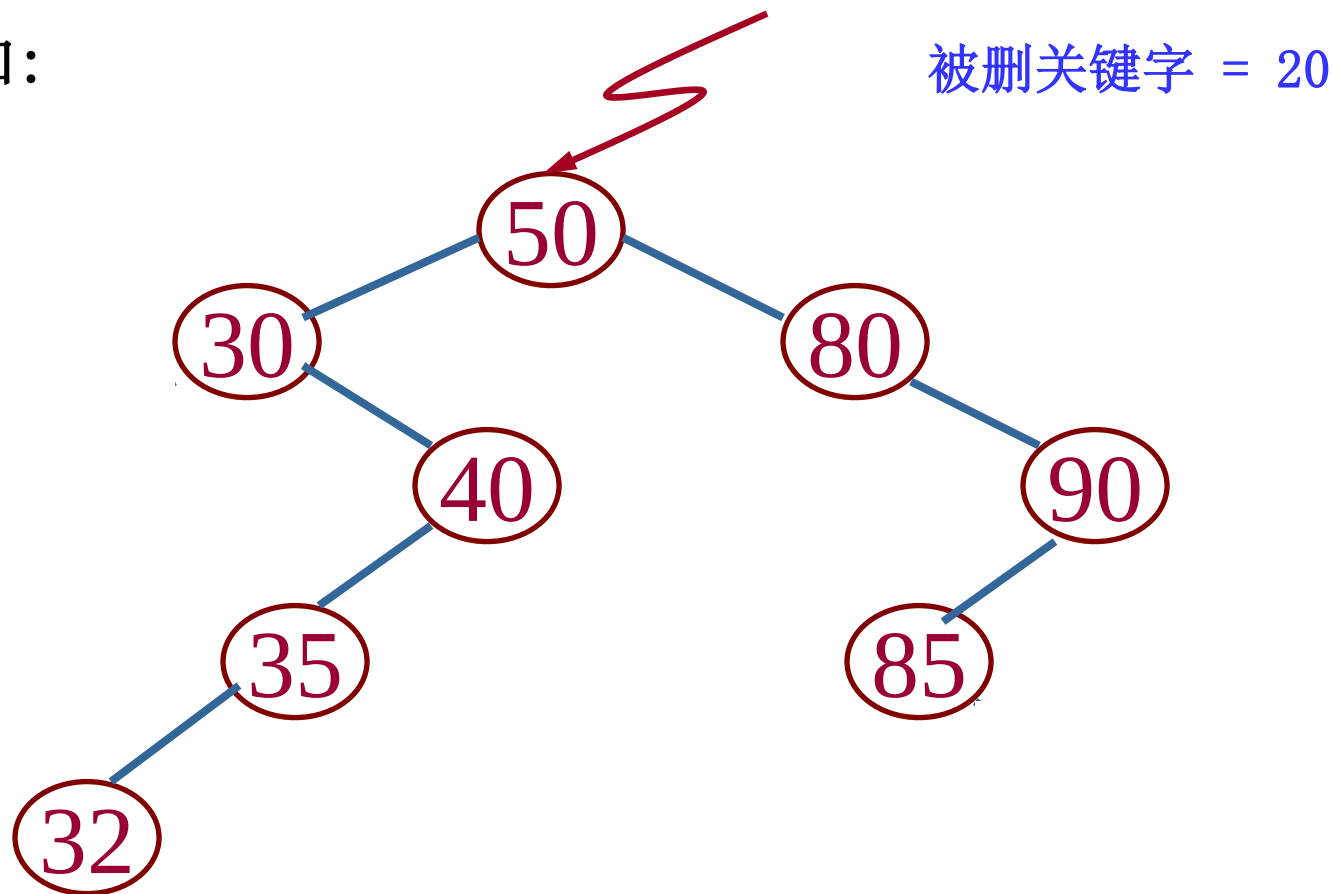
为保证在执行删除后，树的搜索性能不至于降低，还需要**防止重新链接后树的高度增加**

可分三种情况讨论：

- (1) 被删除的结点是叶子；
- (2) 被删除的结点只有左子树或者只有右子树；
- (3) 被删除的结点既有左子树，也有右子树。

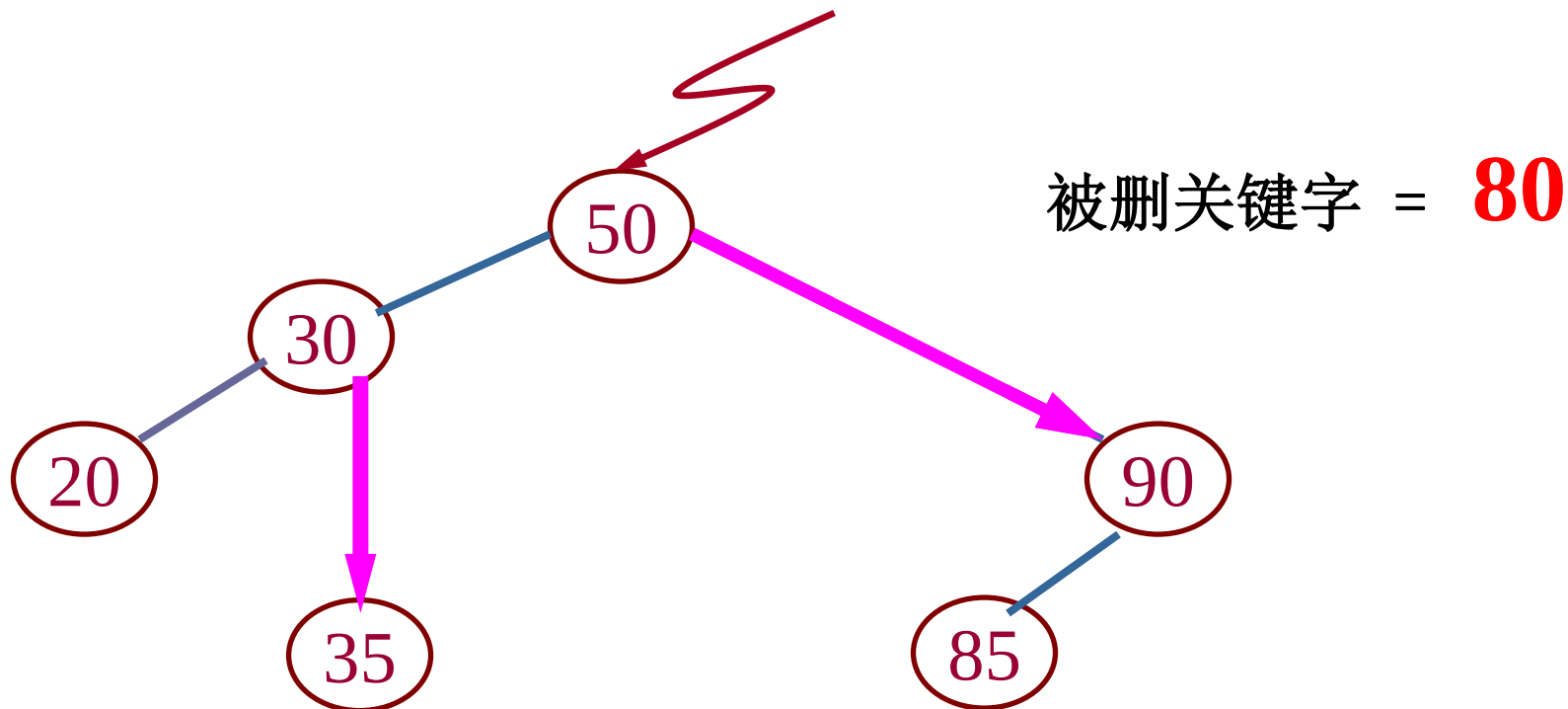
(1) 被删除的结点是叶子结点

例如：



其双亲结点中相应指针域的值改为“空”

(2) 被删除的结点只有左子树或者只有右子树



其双亲结点的相应指针域的值改为“指向被删除结点的左子树或右子树”。

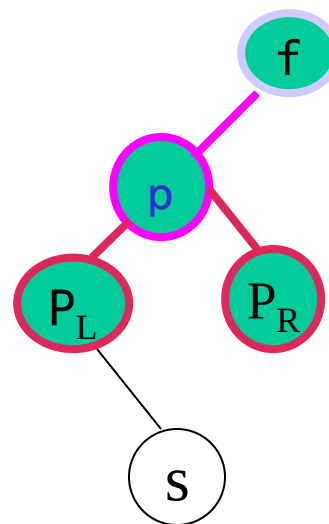
(3) 被删除的结点既有左子树，也有右子树

分析：

设删除前的中序遍历序列为：

.... P_L s p P_R f

// p 的直接前驱是 s , s 是 p 左子树最右下方的结点



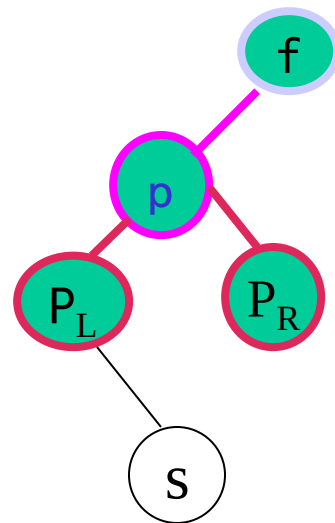
(3) 被删除的结点既有左子树，也有右子树

删除 p 后，其它元素的相对位置不变。有两种解决方法：

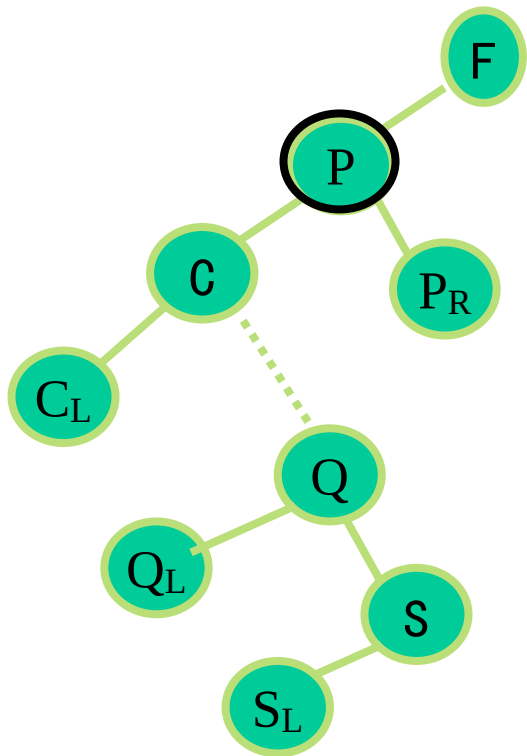
1、令 p 的左子树为 f 的左子树， p 的右子树接为 s 的右子树； //即 $f_L = P_L$ ；

$$S_R = P_R$$

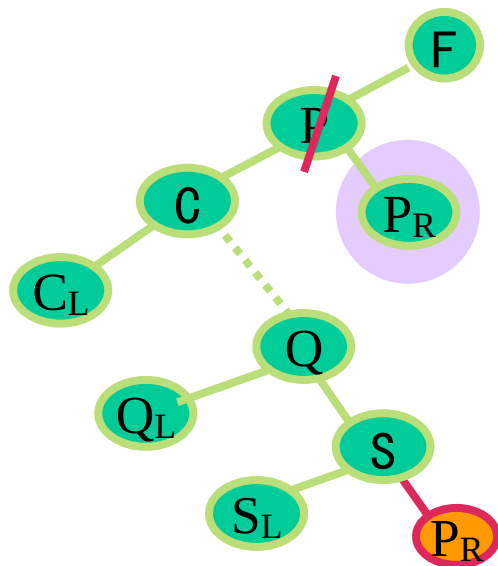
2、直接令 $*s$ 代替 $*p$ // $*s$ 为 $*p$ 左子树最右下方的结点



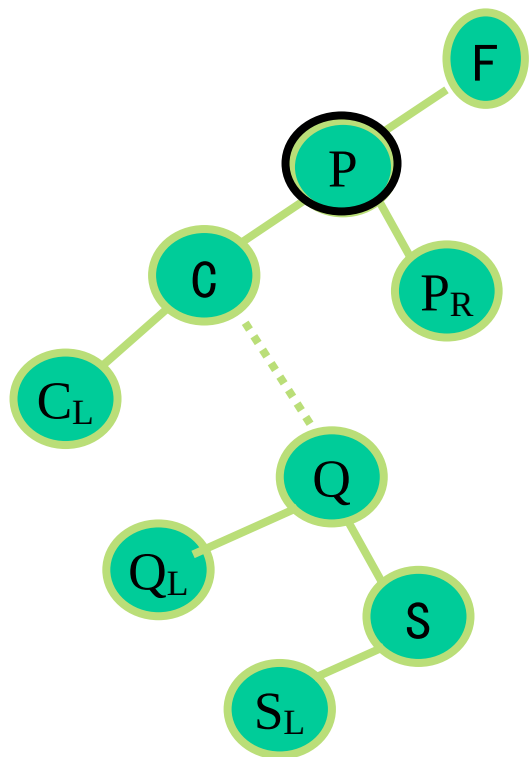
例：请从下面的二叉排序树中删除结点P。



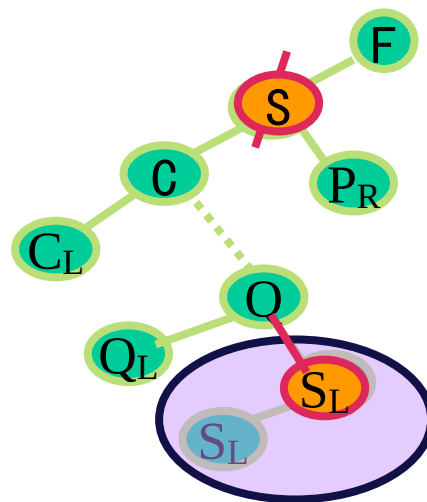
方法1



例：请从下面的二叉排序树中删除结点P。



方法2



算法描述如下：

```
Status DeleteBST (BiTree &T, KeyType key ) {  
    // 若二叉排序树 T 中存在其关键字等于 key 的  
    // 数据元素，则删除该数据元素结点，并返回  
    // 函数值 TRUE，否则返回函数值 FALSE  
    if (!T)  
        return FALSE;    // 不存在关键字等于key的数据元素  
    else {    ...    }  
} // DeleteBST
```

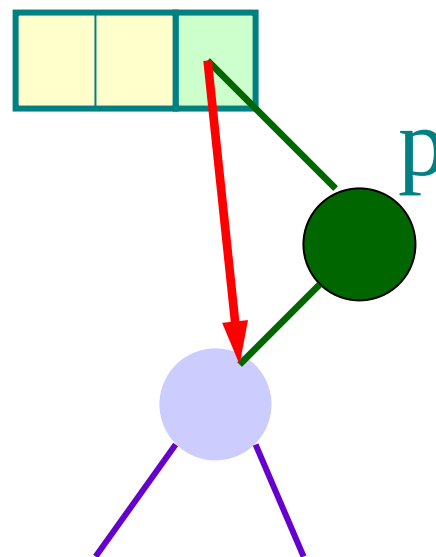
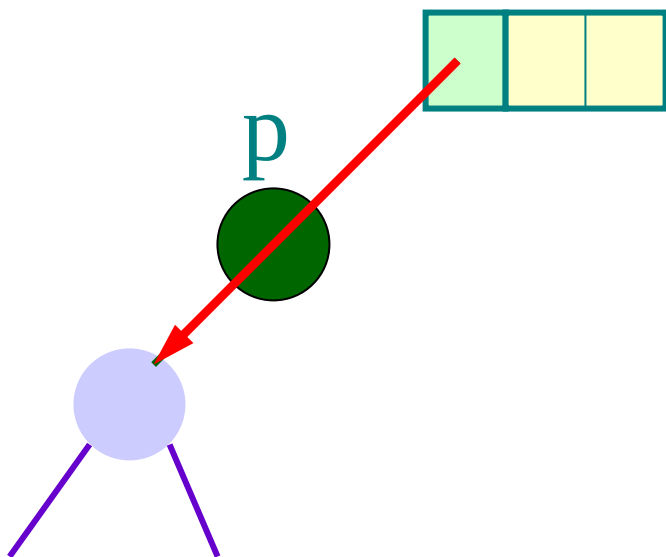
```
if ( EQ (key, T->data.key) )  
    { Delete (T); return TRUE; } // 找到关键字等于key的数据元素  
else if ( LT (key, T->data.key) )  
    DeleteBST ( T->lchild, key ); // 继续在左子树中进行查找  
else  
    DeleteBST ( T->rchild, key ); // 继续在右子树中进行查找
```

其中删除操作过程如下所描述：

```
void Delete ( BiTree &p )  
{ // 从二叉排序树中删除结点 p,  
  // 并重接它的左子树或右子树  
  if (!p->rchild) { ... }  
  else if (!p->lchild) { ... }  
  else { ... }  
} // Delete
```

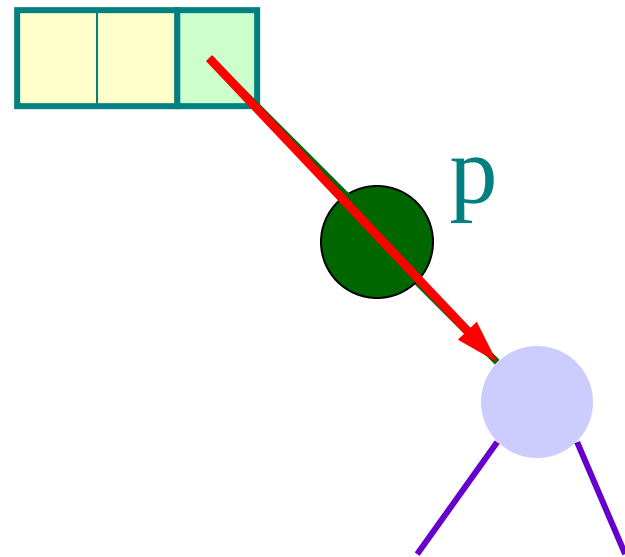
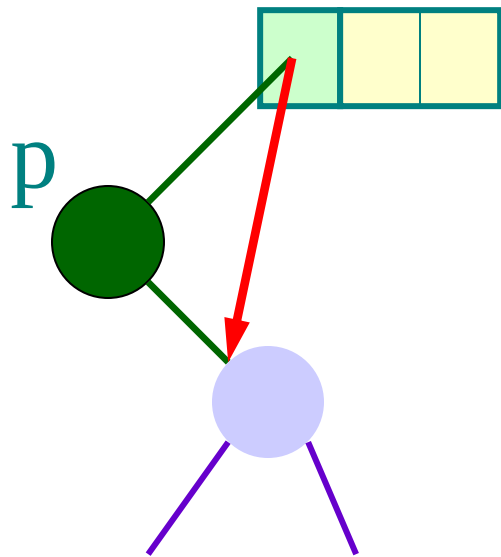

//右子树为空树则只需重接它的左子树

```
q = p; p = p->lchild; free(q);
```



//左子树为空树只需重接它的右子树

```
q = p; p = p->rchild; free(q);
```



// 左右子树均不空

```
q = p; s = p->lchild;
```

// 转左，然后向右走向尽头，s 指向被删结点的前驱

```
while (s->rchild) { q = s; s = s->rchild; }
```

```
p->data = s->data;
```

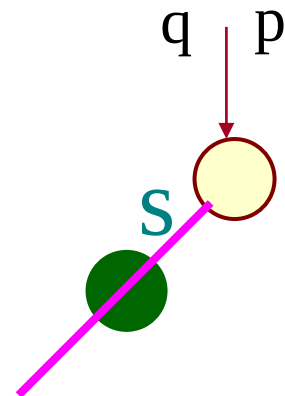
```
if (q != p )
```

```
    q->rchild = s->lchild;
```

```
else
```

```
    q->lchild = s->lchild; // 重接*q的左子树
```

```
free(s);
```



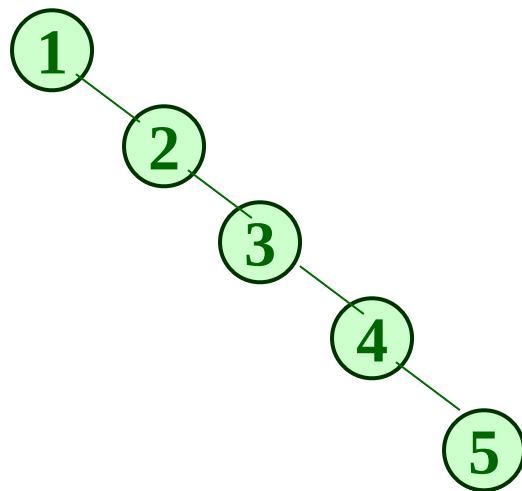
5. 查找性能的分析

对于每一棵特定的二叉排序树，均可按照平均查找长度的定义来求它的 ASL 值，显然，由值相同的 n 个关键字，构造所得的不同形态的各棵二叉排序树的平均查找长度的值不同，甚至可能差别很大。

例如：

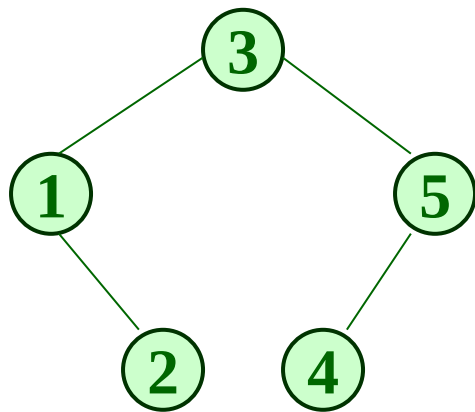
由关键字序列**1, 2, 3, 4, 5**构造而得的
二叉排序树，

$$\begin{aligned} ASL &= (1+2+3+4+5) / 5 \\ &= 3 \end{aligned}$$



由关键字序列**3, 1, 2, 5, 4**构造而得的
二叉排序树，

$$\begin{aligned} ASL &= (1+2+3+2+3) / 5 \\ &= 2.2 \end{aligned}$$

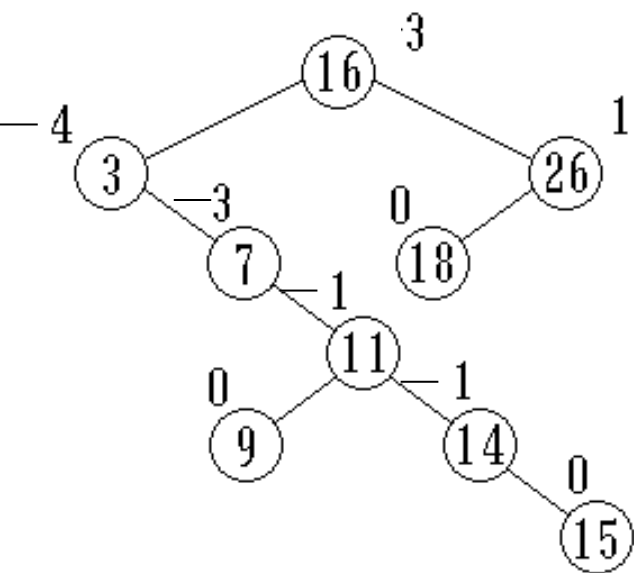


二、二叉平衡树

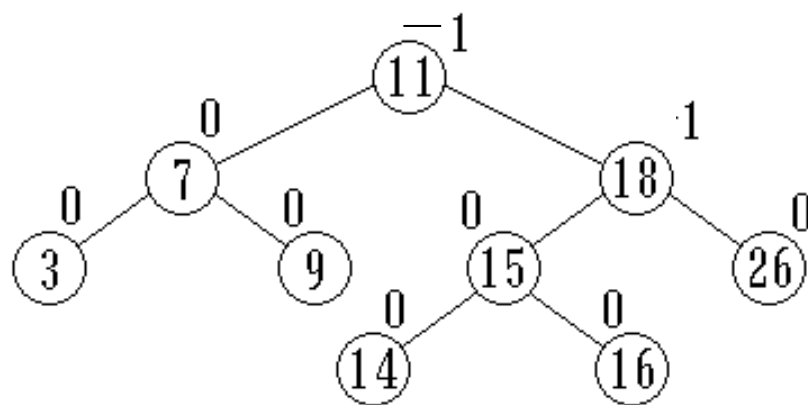
- 何谓“二叉平衡树”？
- *如何构造“二叉平衡树”
- *二叉平衡树的查找性能分析

高度平衡的二叉搜索树 (AVL树)

AVL树的定义:一棵AVL树或者是空树，或者是具有下列性质的二叉搜索树：它的左子树和右子树都是AVL树，且左子树和右子树的高度之差的绝对值不超过1。



高度不平衡的二叉搜索树



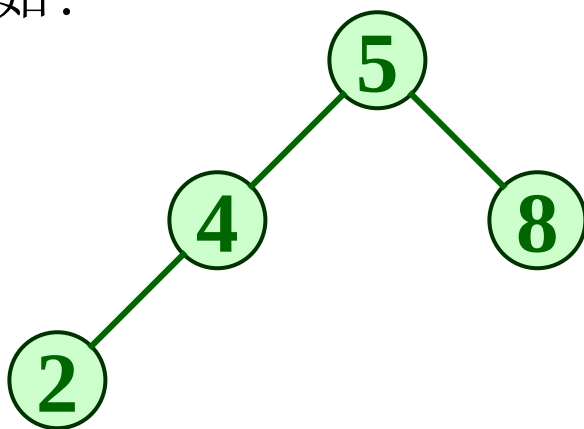
高度平衡的二叉搜索树

二叉平衡树的特点为：

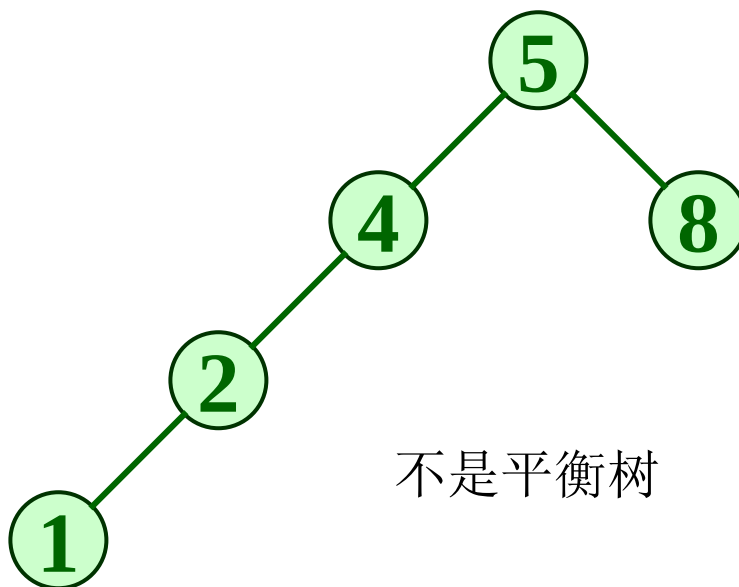
树中每个结点的左、右子树深度之差的绝对值不大于1。

$$|h_L - h_R| \leq 1$$

例如：



是平衡树



不是平衡树

9.1 基本概念

9.2 静态查找表

9.3 动态查找表

9.4 哈希表

一、哈希表的概念

哈希表：即散列存储结构。

散列法存储的基本思想：建立关键码字与其存储位置的对应关系，或者说，由关键码的值决定数据的存储地址。

优点：查找速度极快（ $O(1)$ ），查找效率与元素个数 n 无关！

一、哈希表的概念

例1：若将学生信息按如下方式存入计算机，如：

将2001011810201的所有信息存入V[01]单元；

将2001011810202的所有信息存入V[02]单元；

.....

将2001011810231的所有信息存入V[31]单元。

欲查找学号为2001011810216的信息，便可直接访问V[16]！

例2：有数据元素序列(14, 23, 39, 9, 25, 11)，若规定每个元素 k 的存储地址 $H(k)=k$ ，请画出存储结构图。

$H(k)$ 称为散列函数

根据散列函数 $H(k)=k$ ，可知元素14应当存入地址为14的单元，元素23应当存入地址为23的单元，……，对应散列存储表（哈希表）如下：

地址	...	9	...	11	...	14	...	23	24	25	...	39	...
内容		9		11		14		23		25		39	

地址	...	9	...	11	...	14	...	23	24	25	...	39	...
内容		9		11		14		23		25		39	

讨论：如何进行散列查找？

根据存储时用到的散列函数 $H(k)$ 表达式，迅即可查到结果！

例如，查找 $key=9$ ，则访问 $H(9)=9$ 号地址，若内容为9则成功；
若查不到，应当设法返回一个特殊值，例如空指针或空记录。

明显缺点：空间效率低

Hash查找法

用尽量少的冗余空间
实现 $O(1)$ 的查找效率

哈希方法：选取某个函数，依该函数按关键字计算元素的存储位置并按此存放；查找时也由同一个函数对给定值 k 计算地址，将 k 与地址中内容进行比较，确定查找是否成功。

哈希函数：哈希方法中使用的转换函数称为哈希函数

Hash查找法

哈希表：按上述思想构造的表称为哈希表

冲突：通常关键码的集合比哈希地址集合大得多，因而经过哈希函数变换后，可能将不同的关键码映射到同一个哈希地址上，这种现象称为冲突。

冲突现象举例：

有6个元素的关键码分别为：(14, 23, 39, 9, 25, 11)

选取关键码与元素位置间的函数为 $H(k) = k \bmod 7$

通过哈希函数对6个元素建立哈希表：

0	1	2	3	4	5	6
14		23		39		

$H(14) = 14 \% 7 = 0$

9

25

11

$H(25) = 25 \% 7 = 4$

$H(11) = 11 \% 7 = 4$

6个元素用7个地址应该足够！

有冲突！

在哈希查找方法中，冲突是不可能避免的，只能尽可能减少。哈希方法必须解决以下两个问题：

1) 构造好的哈希函数

- (a) 所选函数尽可能简单，以便提高转换速度；
- (b) 所选函数对关键码计算出的地址，应在哈希地址内集中并大致均匀分布，以减少空间浪费。

在哈希查找方法中，冲突是不可能避免的，只能尽可能减少。哈希方法必须解决以下两个问题：

2) 制定一个好的解决冲突的方案

查找时，如果从哈希函数计算出的地址中查不到关键码，则应当依据解决冲突的规则，有规律地查询其它相关单元。

二、哈希函数的构造方法

要求一： n 个数据原仅占用 n 个地址，虽然散列查找是以空间换时间，但仍希望散列的地址空间尽量小。

要求二：无论用什么方法存储，目的都是尽量均匀地存放元素，以避免冲突。

二、哈希函数的构造方法

1. 直接定址法
2. 除留余数法
3. 乘余取整法
4. 数字分析法
5. 平方取中法
6. 折叠法
7. 随机数法

1、直接定址法

$$\text{Hash}(\text{key}) = a \cdot \text{key} + b \quad (a、b \text{ 为常数})$$

优点：以关键码key的某个线性函数值为哈希地址，不会产生冲突。

缺点：要占用连续地址空间，空间效率低。

1、直接定址法

例：关键码集合为{100, 300, 500, 700, 800, 900},
选取哈希函数为 $\text{Hash}(\text{key})=\text{key}/100$, 则存储结构（哈希表）如下：

0	1	2	3	4	5	6	7	8	9
	100		300		500		700	800	900

2、除留余数法

$\text{Hash}(\text{key}) = \text{key} \bmod p$ (p 是一个整数)

特点：以关键码除以 p 的余数作为哈希地址。

关键：如何选取合适的 p ？

技巧：若设计的哈希表长为 m ，则一般取 $p \leq m$ 且为质数
(也可以是合数，但不能包含小于20的质因子)。

2、除留余数法

为什么要对 p 加限制？

例如：给定一组关键字为：12, 39, 18, 24, 33, 21，若取 $p=9$ ，则他们对应的哈希函数值将为：3, 3, 0, 6, 6, 3

可见，若 p 中含质因子3，则所有含质因子3的关键字均映射到“3的倍数”的地址上，从而增加了“冲突”的可能。

3、数字分析法

选用关键字的某几位组合成哈希地址。

选用原则：各种符号在该位上出现的频率大致相同。

3、数字分析法

例：有一组（例如80个）关键码，其样式如下：

3	4	7	0	5	2	4
3	4	9	1	4	8	7
3	4	8	2	6	9	6
3	4	8	5	2	7	0
3	4	8	6	3	0	5
3	4	9	8	0	5	8
3	4	7	9	6	7	1
3	4	7	3	9	1	9

位号：① ② ③ ④ ⑤ ⑥ ⑦

① 第1、2位均是“3和4”，第3位也只有“7、8、9”，因此，这几位不能用，余下四位分布较均匀，可作为哈希地址选用。

② 若哈希地址取两位（因元素仅80个），则可取这四位中的任意两位组合成哈希地址，也可以取其中两位与其它两位叠加求和后，取低两位作哈希地址。

4、平方取中法

特点：对关键码平方后，按哈希表大小，取中间的若干位作为哈希地址。

理由：因为中间几位与数据的每一位都相关。

例：2589的平方值为6702921，取中间的029为地址

5、折叠法

特点：将关键码自左到右分成位数相等的几部分（最后一部分位数可以短些），然后将这几部分叠加求和，并按哈希表表长，取后几位作为哈希地址。

适用于：每一位上各符号出现概率大致相同的情况。

5、折叠法

方法1：移位法 — 将各部分的最后一位对齐相加。

方法2：间界叠加法 — 从一端向另一端沿分割界来回折叠后，最后一位对齐相加。

例：元素42751896，

用方法1： $\underline{427} + \underline{518} + \underline{96} = 1041$

用方法2： $\underline{427} \ 518 \ \underline{96} \rightarrow 724 + 518 + 69 = 1311$

6、随机数法

$$\text{Hash}(\text{key}) = \text{random}(\text{key})$$

(random为伪随机函数)

适用于：关键字长度不等的情况。

造表和查找都很方便。

小结：构造哈希函数的原则：

- ① 执行速度（即计算哈希函数所需时间）；
- ② 关键字的长度；
- ③ 哈希表的大小；
- ④ 关键字的分布情况；
- ⑤ 查找频率。

三、冲突处理方法

1. 开放定址法（开地址法）
2. 链地址法（拉链法）
3. 再哈希法（双哈希函数法）
4. 建立一个公共溢出区

1、开放定址法（开地址法）

有冲突时就去寻找下一个空的哈希地址，只要哈希表足够大，空的哈希地址总能找到，并将数据元素存入。

1、开放定址法（开地址法）

1) 线性探测法

$$H_i = (\text{Hash}(\text{key}) + d_i) \bmod m \quad (1 \leq i < m)$$

其中：Hash(key)为哈希函数， m 为哈希表长度， d_i 为增量序列1, 2, ..., $m-1$, 且 $d_i = i$

含义：一旦冲突，就找附近（下一个）空地址存入。

例：关键码集为 $\{47, 7, 29, 11, 16, 92, 22, 8, 3\}$,

设：哈希表表长为 $m=11$;

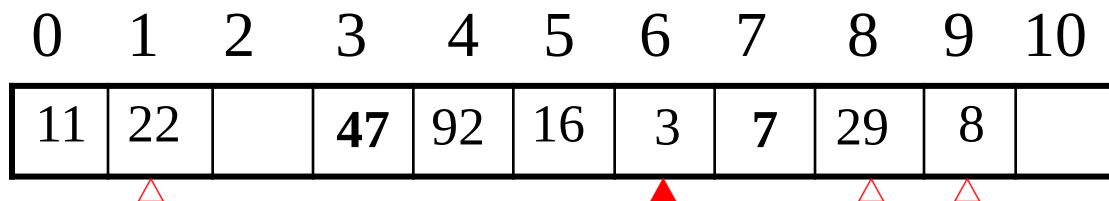
哈希函数为 $\text{Hash}(\text{key}) = \text{key} \bmod 11$;

用线性探测法处理冲突，建哈希表如下：

0	1	2	3	4	5	6	7	8	9	10
11	22		47	92	16	3	7	29	8	



0	1	2	3	4	5	6	7	8	9	10
11	22		47	92	16	3	7	29	8	



- ① 47、7是由哈希函数得到的没有冲突的哈希地址；
- ② $\text{Hash}(29)=7$ ，哈希地址有冲突，需寻找下一个空的哈希地址：由 $H_1=(\text{Hash}(29)+1) \bmod 11=8$ ，哈希地址8为空，因此将29存入。
- ③ 另外，22、8、3同样在哈希地址上有冲突，也是由 H_1 找到空的哈希地址的。

其中3还连续移动了两次（二次聚集）

线性探测法的优点：只要哈希表未被填满，保证能找到一个空地址单元存放有冲突的元素；

线性探测法的缺点：可能使第 i 个哈希地址的同义词存入第 $i+1$ 个哈希地址，这样本应存入第 $i+1$ 个哈希地址的元素变成了第 $i+2$ 个哈希地址的同义词。因此，可能出现很多元素在相邻的哈希地址上“堆积”起来，大大降低了查找效率。

解决方案：可采用二次探测法或伪随机探测法，以改善“堆积”问题。

1、开放定址法（开地址法）

2) 二次探测法

仍举上例，改用二次探测法处理冲突，建表如下：

$$H_i = (\text{Hash}(\text{key}) \pm d_i) \bmod m$$

其中：Hash(key)为哈希函数，m为哈希表长度，m要求是某个4k+3的质数； d_i 为增量序列 $1^2, -1^2, 2^2, -2^2, \dots, q^2$

0	1	2	3	4	5	6	7	8	9	10
11	22	3	47	92	16		7	29	8	

△ ▲ △ △

0	1	2	3	4	5	6	7	8	9	10
11	22	3	47	92	16		7	29	8	

△ ▲ △ △

注：只有3这个关键码的冲突处理与上例不同，

$\text{Hash}(3)=3$ ，哈希地址上冲突，由

$H_1=(\text{Hash}(3)+1^2) \bmod 11=4$ ，仍然冲突；

$H_2=(\text{Hash}(3)-1^2) \bmod 11=2$ ，找到空的哈希地址，存入。

3) 若 d_i =伪随机序列，就称为伪随机探测法

2、链地址法(拉链法)

将具有相同哈希地址的记录链成一个单链表， m 个哈希地址就设 m 个单链表，然后用一个数组将 m 个单链表的表头指针存储起来，形成一个动态的结构。

2、链地址法(拉链法)

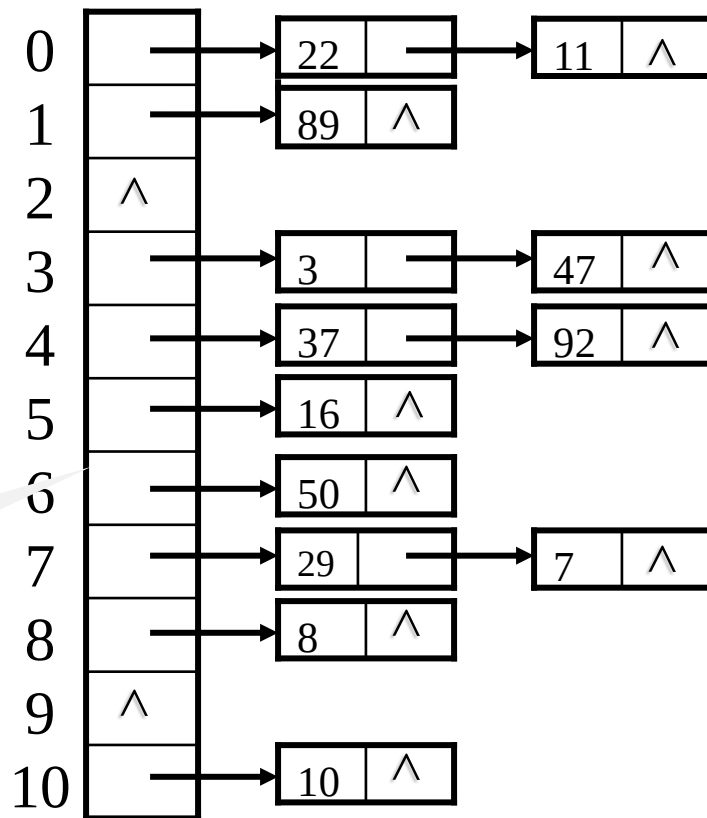
例：设{ 47, 7, 29, 11, 16, 92, 22, 8, 3, 50, 37, 89 }的哈希函数为：

$\text{Hash}(\text{key}) = \text{key} \bmod 11$,

用**拉链法**处理冲突，则建表如右图所示。

有冲突的元素可以插在表尾, 也可以插在表头。

$$ASL = (1 \times 9 + 2 \times 4) / 13 = 17 / 13 \approx 1.3$$



3、再哈希法（双哈希函数法）

$$H_i = RH_i(\text{key}) \quad i = 1, 2, \dots, k$$

RH_i 均是不同的哈希函数，当产生冲突时就计算另一个哈希函数，直到冲突不再发生。

优点：不易产生聚集；

缺点：增加了计算时间。

4. 建立一个公共溢出区

除设立哈希基本表外，另设立一个溢出向量表。

所有关键字和基本表中关键字为同义词的记录，不管它们由哈希函数得到的地址是什么，一旦发生冲突，都填入溢出表。

四、哈希表的查找及分析

//--- 开放定址哈希表的存储结构 ---

```
int hashsize[] = { 997, ... }; //哈希表容量
typedef struct {
    ElemType *elem; // 数据元素存储基址，动态分配数组
    int count;      // 当前数据元素个数
    int sizeindex;  // hashsize[sizeindex]为当前容量
} HashTable;
```

算法9.17

```
Status SearchHash (HashTable H, KeyType K, int &p, int &c) {  
    // 在开放定址哈希表H中查找关键码为K的记录。若查找成功, // 以p指示待查  
    // 数据在表中位置; 否则, 以p指示插入位置  
  
    p = Hash(K);    // 求得哈希地址  
  
    while ( H.elem[p].key != NULLKEY && // 该位置有记录  
           !EQ(K, H.elem[p].key)) // 并且关键字不相等  
        collision(p, ++c);    // 求得下一探查地址 p  
  
    if (EQ(K, H.elem[p].key)) return SUCCESS;  
        // 查找成功, 返回待查数据元素位置 p  
  
    else return UNSUCCESS;    // 查找不成功  
}  
// SearchHash
```

算法9.18

```
Status InsertHash (HashTable &H, Elemtype e){  
    c = 0;  
    // 表中已有与 e 有相同关键字的元素  
    if ( HashSearch ( H, e.key, p, c ) == SUCCESS )  
        return DUPLICATE;  
    // 冲突次数 c 未达到上限, (阈值 c可调)  
    else if ( c < hashsize[H.sizeindex]/2 ) {  
        H.elem[p] = e; ++H.count; return OK;} // 插入e  
    else  
        RecreateHashTable(H);    // 重建哈希表  
} // InsertHash
```

四、哈希表的查找及分析

散列函数没有“万能”通式（杂凑法），要根据元素集合的特性而分别构造。

讨论：哈希查找的速度是否为真正的 $O(1)$ ？

不是。由于冲突的产生，使得哈希表的查找过程仍然要进行比较，仍然要以平均查找长度 ASL 来衡量。

四、哈希表的查找及分析

一般地， ASL 依赖于哈希表的装填因子 α ，它标志着哈希表的装满程度。

$$\alpha = \frac{\text{表中填入的记录数}}{\text{哈希表的长度}} \quad 0 \leq \alpha \leq 1$$

α 越大，表中记录数越多，说明表装得越满，发生冲突的可能性就越大，查找时比较次数就越多。

四、哈希表的查找及分析

1) 散列存储的查找效率到底是多少？

答：ASL与装填因子 α 有关！既不是严格的 $O(1)$ ，也不是 $O(n)$

$$ASL \approx 1 + \frac{\alpha}{2}$$

(拉链法)

$$ASL \approx \frac{1}{2} \left(1 + \frac{1}{1-\alpha} \right)$$

(线性探测法)

$$ASL \approx -\frac{1}{\alpha} \ln(1-\alpha)$$

(随机探测法)

应选择一个合适的 α ，以降低ASL的长度

四、哈希表的查找及分析

2) “冲突”是不是特别讨厌？

答：不一定！正因为有冲突，使得文件加密后无法破译！
（单向散列函数不可逆，常用于数字签名和间接加密）。

利用了哈希表性质：源文件稍稍改动，会导致哈希表变动很大。

典型应用：md5散列算法

重点：各种查找法的基本思想和算法实现

主要算法：

静态查找法——顺序查找、折半查找、分块查找

动态查找法——二叉排序树查找

散列表查找——解决冲突：开地址和链地址法

难点：二叉排序树的删除算法

9.3

9.19

9.33